

Java 16 — Records (Final)

Aspect	Details
Java Version	16
Year	2021
Feature	Records — Final
UniqueClassName	Records
JEP	395 — Records
Purpose	Provides a concise way to create classes whose primary purpose is to model immutable data .
Main problem solved	Eliminates repetitive boilerplate such as fields, constructors, accessors, equals(), hashCode(), and toString().
Basic syntax	public record Employee(int id, String name, double salary) { }
Generated members	Private final fields, canonical constructor, accessor methods (id(), name()), equals(), hashCode(), and toString().
Mutability	The record's components are final; however, referenced objects themselves are not automatically deeply immutable .
Inheritance	A record cannot extend another class. It implicitly extends java.lang.Record.
Interfaces	A record can implement interfaces .
Methods	You can add methods, static members, validation, constructors, and implement interfaces.
Java 16 status	Final / permanent feature . It was previewed in Java 14 and Java 15.

Example

```
public record Employee(  
    int id,  
    String name,  
    String department  
) {  
}
```

Instead of writing a traditional class with fields, constructor, getters, equals(), hashCode(), and toString(), the record provides the data-carrier behavior automatically.

Advantages of Records

Advantage	Explanation	Real-time benefit
Less boilerplate	Removes repetitive constructors, accessors, equals(), hashCode(), and toString().	DTOs become much smaller and easier to maintain.
Immutable by design	Record components are final and cannot be reassigned after construction.	Useful for request/response data and value objects.
Readable	The class declaration immediately shows what data it contains.	Developers can understand DTO structure quickly.
Automatic equals() / hashCode()	Equality is based on record components.	Useful for value objects, map keys, sets, and caching.
Automatic toString()	Produces a useful representation of the record contents.	Helpful for logging and debugging.
Good for DTOs	Excellent fit for data-transfer objects.	REST API request/response models can be concise.
Good for value objects	Naturally represents values such as coordinates, money-like structures, or ranges.	Reduces accidental mutation.
Supports validation	Compact constructors can validate input.	Invalid objects can be rejected during creation.
Works with pattern matching	Records integrate well with modern Java pattern-matching features.	Makes data extraction cleaner in newer Java versions.
Strongly communicates intent	A record says: "this type primarily represents data."	Helps distinguish data carriers from behavior-heavy domain objects.

Disadvantages of Records

Disadvantage	Explanation	Impact
Not deeply immutable	final references do not make referenced mutable objects immutable.	record <code>Order(List<Item> items)</code> can still expose a mutable list.
Cannot extend a class	Records already extend <code>java.lang.Record</code> .	Cannot inherit from a custom base class.
Not suitable for mutable state	Components cannot be reassigned.	Poor fit for entities whose state changes frequently.
Limited customization of generated members	Record semantics are intentionally constrained.	A normal class is better when object identity/state behavior is complex.
Accessors are different from JavaBean getters	<code>name()</code> rather than <code>getName()</code> .	Some older frameworks/tools may expect <code>getName()</code> .
Framework compatibility	Older libraries may not fully understand records.	Can matter in legacy applications.
Serialization considerations	Java serialization has specific record semantics and may differ from traditional classes.	Requires care in legacy serialization systems.
Poor fit for JPA entities	JPA entities generally need mutable lifecycle state, proxies, and framework-specific construction/access patterns.	Prefer normal classes for typical JPA entities.
No explicit no-argument constructor by default	The canonical constructor is generated from components.	Some frameworks requiring a no-arg constructor may have issues.
Not ideal for behavior-heavy objects	Records are primarily data carriers.	A regular class communicates richer domain behavior better.

When to Use Records

Scenario	Use Record?	Example
REST API response DTO	✓ Yes	record UserResponse(long id, String name)
REST API request DTO	✓ Yes	record CreateUserRequest(String name, String email)
Database query projection	✓ Yes	record UserSummary(long id, String name)
Value object	✓ Yes	record Money(BigDecimal amount, String currency)
Coordinates	✓ Yes	record Point(double latitude, double longitude)
Configuration snapshot	✓ Yes	record DatabaseConfig(String host, int port)
Message/event data	✓ Yes	record PaymentCompleted(String paymentId, BigDecimal amount)
Map key	✓ Yes	record Coordinate(int x, int y)
Temporary result object	✓ Yes	record CalculationResult(double value, boolean successful)
Immutable application state	✓ Yes	record ApplicationState(String status, long version)
Pattern-matching data model	✓ Yes	Records work particularly well with modern pattern matching.

Example: REST DTO

```
public record UserResponse(
    Long id,
    String name,
    String email
) {
}
```

A controller can return this directly:

```
@GetMapping("/users/{id}")
public UserResponse getUser(long id) {
    return new UserResponse(id, "Anurag", "user@example.com");
}
```

}

When NOT to Use Records

Scenario	Avoid Record?	Why	Better Alternative
JPA entity	✗ Usually	Entity state changes and ORM frameworks need special lifecycle behavior.	Normal class
Highly mutable object	✗ Yes	Record components cannot be reassigned.	Normal class with setters/methods
Complex inheritance hierarchy	✗ Yes	Records cannot extend custom classes.	Abstract class / normal class
Object with extensive behavior	✗ Usually	A record primarily communicates data-carrier semantics.	Normal class
Framework requiring JavaBean setters	✗ Usually	Records don't provide setters.	JavaBean-style class
Deeply mutable object graph	✗ Usually	Records don't automatically make nested objects immutable.	Immutable class / defensive copies
Objects with identity-based lifecycle	✗ Usually	Records are value-oriented.	Entity class
Legacy serialization/framework constraints	✗ Maybe	Older infrastructure may expect traditional POJOs.	POJO
Need multiple inheritance through a base class	✗ Yes	Records cannot extend another class.	Normal class / composition

Real-Time Use Cases

Real-world application	Record example	Why Record fits

Banking API	record AccountSummary(String accountNo, BigDecimal balance)	Immutable response data
E-commerce	record ProductDto(long id, String name, BigDecimal price)	API/data-transfer model
Payment system	record PaymentResult(String transactionId, boolean successful)	Result/value object
Microservices	record OrderCreatedEvent(long orderId, Instant createdAt)	Event payload
Location services	record Coordinates(double latitude, double longitude)	Natural value object
Reporting	record SalesSummary(String region, BigDecimal revenue)	Query/report projection
Authentication	record UserPrincipal(long id, String username)	Immutable identity data
Caching	record CacheKey(String tenantId, String resourceId)	Reliable value-based equality
Configuration	record ServerConfig(String host, int port, boolean ssl)	Immutable configuration
Search results	record SearchResult(long id, String title, double score)	Simple data carrier

Record with Validation

Records can contain a **compact constructor**:

```
public record Employee(  
    int id,
```

```

    String name,
    double salary
) {
    public Employee {
        if (id <= 0) {
            throw new IllegalArgumentException("Invalid employee ID");
        }

        if (salary < 0) {
            throw new IllegalArgumentException("Salary cannot be negative");
        }
    }
}

```

This is particularly useful when the record represents a value that must always satisfy certain invariants.

Record vs Traditional Class

Feature	Record	Traditional Class
Boilerplate	Very low	Potentially high
Final components	✓	Optional
Automatic constructor	✓	Must normally write
Accessors	✓	Must normally write
equals()	✓ Component-based	Must implement/override
hashCode()	✓ Component-based	Must implement/override
toString()	✓	Must implement/override
Setters	✗	✓ Possible
Extends custom class	✗	✓
Implements interfaces	✓	✓
Mutable state	✗ Not suited	✓
Deep immutability	✗ Not automatic	Depends on implementation
Best purpose	Data/value carrier	General-purpose object

Framework flexibility	Moderate	High
Domain behavior	Limited/moderate	Excellent

Alternate Features to Records

There isn't one universal replacement; the appropriate alternative depends on **why you are using the record**.

Requirement	Record	Alternative	When Alternative Is Better
Simple immutable data carrier	✓ Best choice	Final class	When you need more customization
Mutable DTO	✗	JavaBean/POJO	When setters are required
Complex domain object	□	Normal class	When behavior and lifecycle dominate
ORM/JPA entity	✗ Usually	JPA entity class	When persistence framework manages object state
Immutable object with deep immutability	□	Immutable class	When nested objects must also be immutable
Existing inheritance hierarchy	✗	Normal class	When extending a base class is required
Legacy framework compatibility	□	POJO	When framework expects JavaBean conventions
Value object	✓ Excellent	Final class	When custom inheritance/framework behavior is needed
API DTO	✓ Excellent	POJO	Mainly when tooling requires JavaBean accessors

Practical Rule

If your object...	Recommendation
Primarily carries data	☐ Use a Record
Represents a value	☐ Use a Record
Should not change after construction	☐ Use a Record
Is an API request/response DTO	☐ Use a Record
Is an event/message payload	☐ Use a Record
Needs setters/mutable state	☐ Use a normal class
Must extend another class	☐ Use a normal class
Is a typical JPA entity	☐ Use a normal class
Has complex lifecycle/identity	☐ Use a normal class
Requires deep immutability	☐ Record + defensive copies, or an immutable class

Bottom line

Records are best viewed as **concise, value-oriented data models**—not as a replacement for every Java class. For modern Java applications, they are especially valuable for **DTOs, API models, query projections, events, configuration snapshots, and value objects**.

Java 16 — Pattern Matching for instanceof (Final)

Aspect	Details
Java Version	16
Year	2021
Feature	Pattern Matching for instanceof — Final
UniqueClassName	InstanceofFinal

JEP	394 — Pattern Matching for instanceof
Status	Final / Permanent feature in Java 16
Main purpose	Combines type checking and casting into a single operation.
Problem solved	Eliminates explicit casts after a successful instanceof check.
Traditional syntax	if (obj instanceof String) { String s = (String) obj; }
Java 16 syntax	if (obj instanceof String s) { }
Main benefit	Less boilerplate, fewer opportunities for casting errors, and more readable type-dependent code.

1. Basic Comparison

Traditional instanceof	Pattern Matching for instanceof
java if (obj instanceof String) { String s = (String) obj; System.out.println(s.length()); }	java if (obj instanceof String s) { System.out.println(s.length()); }
Requires explicit cast	Cast is implicit
Two operations: test + cast	One operation: test + pattern variable
More verbose	More concise
Cast can potentially be repeated	No repeated cast required

The pattern variable s is automatically initialized only when the instanceof test succeeds.

2. Advantages

Advantage	Explanation	Example / Benefit
Less boilerplate	Eliminates explicit casting after instanceof.	if (x instanceof String s)
Improved readability	Type check and conversion are expressed together.	Easier to understand conditional logic.

Type safety	The compiler understands the type of the pattern variable.	s.length() is known to be valid.
Reduces casting mistakes	No need to manually repeat the cast.	Avoids (String) obj everywhere.
Better maintainability	Fewer repeated expressions mean less code to modify.	Useful in large conditional code.
Pattern variable scope	Variable is available only where the compiler knows the match succeeded.	Prevents accidental use outside the valid scope.
Works with boolean expressions	Pattern variables can flow through logical conditions.	if (x instanceof String s && !s.isBlank())
Supports negation	Can be used with !instanceof while respecting flow scoping.	if (!(x instanceof String s)) return;
Works well with polymorphism	Makes type-specific behavior easier to express.	Processing different object types.
Foundation for modern pattern matching	Provides a basis for later pattern-matching features such as switch.	Cleaner modern Java code.

3. Disadvantages

Disadvantage	Explanation	Impact
Still relies on type checks	Frequent instanceof checks can indicate poor polymorphic design.	May be better to use interfaces/overriding.
Can encourage procedural type logic	Large chains of instanceof can become difficult to maintain.	if/else if chains can grow quickly.
Not a substitute for polymorphism	It does not eliminate the need for proper object-oriented design.	Behavior may belong in the object itself.
Limited to matching a	It does not automatically	Visitor/polymorphism may be

type	solve complex object-dispatch requirements.	better.
Scope can initially be confusing	Flow scoping determines where the pattern variable exists.	Developers unfamiliar with the feature may need practice.
Requires Java 16+ for final feature	Older Java versions cannot use the final syntax directly.	Relevant to legacy applications.
Potentially verbose for many types	Multiple type checks can still produce long code.	Consider polymorphism or switch patterns in newer Java.
Can hide design problems	Excessive type inspection may suggest classes have too much knowledge about subclasses.	Can weaken encapsulation.

4. When to Use

Situation	Use?	Reason
Checking an object and immediately using its type	✓ Yes	This is the primary use case.
Processing heterogeneous objects	✓ Yes	Simplifies type-specific processing.
Validating API input	✓ Yes	Convenient for type-sensitive validation.
Parsing generic/object-based data	✓ Yes	Avoids repeated casts.
Working with legacy APIs returning Object	✓ Yes	Makes old APIs easier to consume.
Small number of type checks	✓ Yes	Clear and concise.
One-off type-specific operation	✓ Yes	Avoids unnecessary abstraction.
Debugging/logging	✓ Yes	Convenient for conditional

different object types		inspection.
Type-specific equality/comparison logic	☐ Sometimes	Useful, but consider polymorphism/value objects first.
Complex business rules based on many types	☐ Usually avoid	Polymorphism or pattern switch may be cleaner.

5. When NOT to Use

Situation	Avoid?	Better approach
Same type checks repeated throughout the application	✗	Polymorphism / interface
Every subclass has different behavior	✗	Method overriding
Large if/else instance of chain	✗	Polymorphism or modern switch patterns
Object hierarchy is under your control	☐	Prefer polymorphic behavior where appropriate
Behavior naturally belongs to the object	✗	Put behavior in the class
Need extensible business logic	✗	Strategy / polymorphism
Need to handle many unrelated types	☐	Consider a dispatcher, visitor, or pattern switch
Simple instance of check with no need for cast	☐	Ordinary instance of may be sufficient

6. Real-Time Use Cases

Real-time use case	Example	Why useful
REST/API processing	Different request payload	Avoids explicit casts.

	types	
Payment processing	CardPayment, UPIPayment, CashPayment	Allows type-specific inspection.
E-commerce	Book, Electronic, Clothing products	Convenient when a common API returns different types.
Logging	Inspecting different event objects	Extracts type-specific information easily.
Message processing	Different event payload classes	Useful for heterogeneous message objects.
UI event handling	Different event subclasses	Simplifies event-specific handling.
File processing	Different file metadata objects	Type-safe access after matching.
Legacy APIs	APIs returning Object	Reduces repetitive casting.
Validation frameworks	Different input types	Type-specific validation becomes concise.
AST/compiler processing	Different syntax node types	Useful for examining nodes returned through a common type.

7. Real-Time Example — Payment Processing

Traditional Java

```
public void process(Object payment) {

    if (payment instanceof CreditCardPayment) {
        CreditCardPayment p = (CreditCardPayment) payment;
        processCard(p);
    }
    else if (payment instanceof UPIPayment) {
        UPIPayment p = (UPIPayment) payment;
        processUPI(p);
    }
}
```

Java 16

```
public void process(Object payment) {
```

```

    if (payment instanceof CreditCardPayment p) {
        processCard(p);
    }
    else if (payment instanceof UPIPayment p) {
        processUPI(p);
    }
}

```

The Java 16 version removes the redundant cast.

8. Real-Time Example — API Response Processing

Suppose a legacy service returns Object:

```

public void handleResponse(Object response) {

    if (response instanceof String s) {
        System.out.println("Message: " + s);
    }
    else if (response instanceof Integer i) {
        System.out.println("Status code: " + i);
    }
    else if (response instanceof ErrorResponse error) {
        System.out.println("Error: " + error.message());
    }
}

```

The pattern variable has the correct type immediately:

```

Object
├── String → s
├── Integer → i
└── ErrorResponse → error

```

No explicit casts are required.

9. Flow Scoping

One of the important aspects of the feature is **flow scoping**.

```

if (obj instanceof String s && s.length() > 5) {
    System.out.println(s);
}

```

```
}
```

Here `s` is available in the right-hand side of `&&` because Java knows that the first condition succeeded.

Another example:

```
if (!(obj instanceof String s)) {  
    return;  
}
```

```
System.out.println(s.length());
```

After the return, Java knows that `obj` must be a `String`, so `s` is available.

This is safer than manually declaring and casting variables.

10. Pattern Matching vs Polymorphism

Requirement	Pattern Matching	Polymorphism
One or two occasional type checks	☐ Excellent	☐ May be unnecessary
Legacy API returns Object	☐ Excellent	☐ Difficult
Type-specific external processing	☐ Excellent	☐ Depends
Each class owns its own behavior	☐	☐ Better
Many subclasses	☐	☐ Usually better
Frequently changing type hierarchy	☐	☐ Usually better
Extensible business rules	☐	☐
Small utility method	☐	☐
Large domain model	☐	☐

Example where polymorphism is better

Instead of:

```
if (shape instanceof Circle c) {  
    calculateCircleArea(c);  
}  
else if (shape instanceof Rectangle r) {  
    calculateRectangleArea(r);  
}
```

Prefer:

```
interface Shape {  
    double area();  
}
```

and:

```
class Circle implements Shape {  
    public double area() {  
        // ...  
    }  
}
```

```
class Rectangle implements Shape {  
    public double area() {  
        // ...  
    }  
}
```

Then simply:

```
double area = shape.area();
```

Rule: If the behavior naturally belongs to the object, prefer **polymorphism**. If external code genuinely needs to inspect the object's type, pattern matching is often appropriate.

11. Alternate Features

Requirement	instanceof Pattern Matching	Alternative
Simple type check + cast	☐ instanceof pattern	Traditional instanceof + cast
Behavior belongs to object	☐	Polymorphism / method overriding
Many possible object	☐	Pattern matching for switch in

types		newer Java
Fixed set of object variants	☐	Sealed classes + pattern matching
Dispatch based on type	☐	Visitor pattern
Different algorithms	☐	Strategy pattern
Object creation based on type	✗	Factory pattern
Legacy Java application	☐	Traditional instanceof + cast

Most important alternative: Polymorphism

```
interface Payment {
    void process();
}
```

Instead of:

```
if (payment instanceof CreditCardPayment p) {
    p.process();
}
```

you can simply use:

```
payment.process();
```

when the design allows the behavior to be owned by the Payment hierarchy.

12. Overall Evaluation

Category	Rating	Assessment
Readability	★★★★★	Very concise and clear
Boilerplate reduction	★★★★★	Eliminates explicit casts
Type safety	★★★★★	Compiler manages the pattern variable
Maintainability	★★★★★	Less repetitive code

Performance	☆☆☆☆☆	Primarily a language-level readability feature; don't expect major performance gains
Object-oriented design	☆☆☆☆☆	Good when type inspection is genuinely needed
Framework compatibility	☆☆☆☆☆	Generally good with modern Java tooling
Legacy compatibility	☆☆☆☆☆	Requires Java 16+ syntax
Best use	Type check + immediate use	Excellent
Main risk	Overusing type checks	Can indicate missing polymorphism

Bottom line

Pattern Matching for instanceof in Java 16 is an excellent final feature when you genuinely need to check an object's type and then operate on that specific type. Its biggest benefit is cleaner, safer code rather than raw performance improvement.

A good rule is:

Use instanceof Type variable for occasional type inspection; use polymorphism when the object itself should own the behavior.